Java Spring – REST & HTTP

Dr. Öğr. Üyesi Alper Talha KARADENİZ

REST Nedir?

- REST (Representational State Transfer), iki uygulama arasındaki iletişimi kurmak için en sık kullanılan yöntemlerden biridir ve sunucunun istemciye sunduğu işlevlere belirli uç noktalar (endpoints) aracılığıyla erişim sağlar.

Nasıl Çalışır?

- **İletişim Modeli:** REST servisleri, uygulamalar arası veri alışverişini sağlamak için HTTP protokolünü kullanır. Bir web istemcisinin backend ile konuşması, bir mobil uygulamanın sunucuyla iletişimi veya iki backend servisinin birbiriyle veri paylaşması için kullanılır.
- **Veri Formatı:** Web sitelerinden temel farkı, REST servislerinin HTML yerine genellikle JSON veya XML gibi veri odaklı formatlarla yanıt vermesidir.
- **Uç Noktalar (Endpoints):** Bir REST uç noktası, temelde belirli bir HTTP yöntemine (GET, POST, PUT, DELETE vb.) ve bir yola (path) eşlenmiş bir denetleyici (controller) eylemidir.

Spring Framework'te REST

- Spring, REST servislerini geliştirmek için web uygulamalarında kullandığı Spring MVC mekanizmasını kullanır, ancak bazı önemli farklar vardır:
- **View Resolver'ın Atlanması:** Standart web uygulamalarının aksine, REST servislerinde Spring MVC'nin "View Resolver" (Görünüm Çözücü) bileşeni kullanılmaz.
- **Doğrudan Yanıt:** Denetleyici eylemi tarafından döndürülen veriler, herhangi bir HTML şablonuna yerleştirilmeden doğrudan HTTP yanıt gövdesine (response body) yazılır ve istemciye gönderilir.

Spring Framework'te REST

- **@RestController:** Spring'de bir sınıfı REST denetleyicisi olarak işaretlemek için @RestController anotasyonu kullanılır. Bu anotasyon, @Controller ve @ResponseBody anotasyonlarının birleşimidir ve Spring'e tüm metodların yanıtlarının doğrudan istemciye gönderileceğini bildirir.

HTTP Metotları

- HTTP metotları (veya fiilleri), bir istemcinin (örneğin bir tarayıcı veya mobil uygulama) sunucudaki belirli bir kaynakla ne yapmak istediğini ifade eden komutlardır. Spring Framework, bu metotları karşılamak için özel anotasyonlar (örneğin @GetMapping, @PostMapping) sunar.

GET (Veri Okuma)

- **Amacı:** Sunucudan veri veya kaynak almak için kullanılır.
- **Özelliği:** Sunucu tarafındaki veriyi değiştirmez, sadece mevcut bilgiyi istemciye iletir.
- **Spring Karşılığı:** @GetMapping anotasyonu ile yönetilir.

POST (Veri Ekleme)

- Amacı: Sunucuya yeni bir veri göndermek veya yeni bir kaynak oluşturmak için kullanılır.
- Örnek: Bir web formunu doldurup "Gönder" butonuna tıkladığınızda genellikle bir POST isteği oluşur.
- Spring Karşılığı: @PostMapping anotasyonu ile yönetilir. Başarılı bir işlemden sonra genellikle "201 Created" (Oluşturuldu) durumu döner.

PUT ve PATCH (Veri Güncelleme)

- PUT: Mevcut bir kaynağı tamamen değiştirmek (wholesale replacement) için kullanılır. PUT isteğinde, kaynağın tüm alanları gönderilmelidir; eksik bırakılan alanlar genellikle sunucuda null olarak üzerine yazılır.
- PATCH: Bir kaynağın sadece belirli bir kısmını güncellemek için kullanılır. Sadece değişmesi istenen alanlar gönderilir.
- Spring Karşılığı: PUT için @PutMapping, PATCH için @PatchMapping kullanılır.

DELETE (Veri Silme)

- Amacı: Sunucudaki belirli bir kaynağın kaldırılması için kullanılır.
- Özelliği: Başarılı bir silme işleminden sonra sunucu genellikle gövdesi boş olan bir "204 No Content" (İçerik Yok) yanıtı döner.
- Spring Karşılığı: @DeleteMapping anotasyonu ile yönetilir.

@RestController vs @Controller

Özellik	@Controller	@RestController
Kullanım Amacı	HTML sayfaları sunmak (View tabanlı)	Veri odaklı REST API'ler sunmak
Yanıt Şekli	Genellikle bir görünüm (View) adı döner	Nesnenin kendisi (JSON/XML) döner
@ResponseBody	Her metod için manuel eklenmelidir	Sınıf düzeyinde otomatik olarak dahildir
Bileşen Türü	Geleneksel Spring MVC Denetleyicisi	RESTful Web Servis Denetleyicisi

Mapping

- Bu anotasyonlar, Spring MVC içerisinde HTTP isteklerini belirli denetleyici (controller) metotlarına eşlemek için kullanılan özel istek eşleme anotasyonlarıdır. Her biri temel bir HTTP yöntemine (fiiline) karşılık gelir ve genellikle CRUD (Oluşturma, Okuma, Güncelleme, Silme) işlemlerini gerçekleştirmek için kullanılır.
- **@GetMapping:** HTTP GET isteklerini karşılar. Temel amacı sunucudan veri veya kaynak okumaktır. Sunucu tarafındaki veride bir değişiklik yapmaz; sadece mevcut bilgiyi istemciye iletir.

Mapping

- **@PostMapping:** HTTP POST isteklerini karşılar. Tipik olarak sunucuda yeni bir kaynak oluşturmak için kullanılır. Örneğin, bir kayıt formu gönderildiğinde genellikle bu anotasyon kullanılır.
- **@PutMapping:** HTTP PUT isteklerini karşılar. Mevcut bir kaynağı güncellemek için kullanılır; ancak semantik olarak kaynağın tamamen değiştirilmesini (wholesale replacement) ifade eder. Yani istemci, kaynağın tüm alanlarını göndermelidir.

Mapping

- **@PatchMapping:** HTTP PATCH isteklerini karşılar. PUT gibi güncelleme amaçlıdır, ancak kaynağın sadece belirli bir kısmını (kısmi güncelleme) değiştirmek için kullanılır.
- **@DeleteMapping:** HTTP DELETE isteklerini karşılar. Belirli bir kaynağın sunucudan silinmesi işlemini gerçekleştirir.

@RequestParam (Sorgu Parametreleri)

- @RequestParam, verileri HTTP isteğinin URL'sindeki sorgu parametrelerinden (query parameters) okumak için kullanılır.
- Sözdizimi: URL'nin sonuna eklenen ?anahtar=değer formatındaki ifadelerdir (Örn: .../home?color=red).
- Kullanım Amacı: Genellikle filtreleme, arama kriterleri veya sayfalama gibi isteğe bağlı (optional) veriler için tercih edilir.
- Özellikler: Varsayılan olarak zorunludur; eğer istemci bu parametreyi göndermezse uygulama "400 Bad Request" hatası döner. İsteğe bağlı yapmak için required = false özelliği kullanılabilir.

@PathVariable (Yol Değişkenleri)

- @PathVariable, verileri doğrudan URL yolunun (URI path) içinden çıkarmak için kullanılır.
- Sözdizimi: Denetleyici metodunda yol {degisken} şeklinde tanımlanır ve URL'de doğrudan değer olarak yer alır (Örn: /home/blue).
- Kullanım Amacı: Belirli bir kaynağı tanımlayan (ID gibi) zorunlu (mandatory) veriler için idealdir.
- Avantajı: URL'nin daha okunabilir olmasını sağlar ve arama motorları (SEO) tarafından daha kolay indekslenir.

@RequestBody (İstek Gövdesi)

- @RequestBody, HTTP isteğinin gövdesindeki (body) verilerin (genellikle JSON veya XML formatında) bir Java nesnesine dönüştürülerek metoda aktarılmasını sağlar.
- Çalışma Mantığı: Spring, gelen JSON verisini otomatik olarak çözümlemek (deserialize) ve parametre olarak verilen sınıfa (DTO) bağlamak için arka planda mesaj dönüştürücüler (örneğin Jackson) kullanır.
- Kullanım Amacı: Genellikle POST, PUT veya PATCH metotlarında, sunucuya gönderilen büyük miktardaki veya karmaşık yapıdaki verileri (örneğin bir kullanıcı kayıt formu veya sipariş detayları) almak için kullanılır.
- Hata Yönetimi: Eğer gelen veri beklenen Java sınıfı yapısıyla eşleşmiyorsa, Spring "400 Bad Request" yanıtı döner.

Özellik	@RequestParam	@PathVariable	@RequestBody
Veri Kaynağı	URL Sorgu Dizisi (`?id=5`)	URL Yolu (`/users/5`)	İstek Gövdesi (JSON/XML)
Veri Miktarı	Küçük/Basit	Küçük/Basit	Büyük/Karmaşık
Kullanım Senaryosu	Filtreleme, Arama	Kaynak Kimliği (ID)	Kayıt/Güncelleme
Zorunluluk	Varsayılan: Zorunlu	Genellikle Zorunlu	Gövde Gereklidir

JSON → Object dönüşümü (Jackson)

- **JSON'dan Nesneye Dönüşüm (Deserialization):** Bir REST uç noktası (endpoint) @RequestBody anotasyonu ile işaretlenmiş bir parametre aldığında, Spring arka planda Jackson mesaj dönüştürücülerini kullanır,. Bu mekanizma, gelen HTTP istek gövdesindeki JSON verisini otomatik olarak çözümleyip ilgili Java sınıfına (genellikle bir DTO) dönüştürerek metoda aktarır,. Eğer gelen JSON verisi beklenen Java sınıf yapısıyla eşleşmezse, Spring "400 Bad Request" hatası döner (dış kaynak bilgisi).

JSON → Object dönüşümü (Jackson)

- **Nesneden JSON'a Dönüşüm (Serialization):** @RestController ile işaretlenmiş sınıflarda veya @ResponseBody kullanılan metotlarda, bir metodun döndürdüğü Java nesnesi Jackson tarafından otomatik olarak JSON formatına dönüştürülür,. Bu dönüştürülen veri, herhangi bir görünüm şablonuna (view) ihtiyaç duyulmadan doğrudan HTTP yanıt gövdesine yazılır,.